

1. 判断题 (12分)

- a. 两个不同的网页（例如www.mit.edu/research.html和www.mit.edu/students.html）可以在同一个持久连接上传输。
- b. 假设主机A通过TCP连接向主机B发送一个大文件。如果此连接的段的序列号为m，那么后续段的序列号必然是 $m + 1$ 。
- c. TCP段的头部有一个rwnd字段表示拥塞窗口

参考答案

- a. 正确 (true) 。持久连接允许多个网页通过同一TCP连接传输。
- b. 错误 (false) 。后续段的序列号取决于当前段的序列号加上其数据的字节数，不一定是 $m+1$ 。例如如果段m包含1000字节，下一段的序列号是 $m+1000$
- c. 错误 (false) 。

2. 应用层协议 (15分)

TCP/IP 协议族中存在多种应用层协议，不同的应用对可靠性和效率的要求不同，因此选择不同的传输层服务。

请根据以下应用层协议，分析它们各自使用的传输层协议及原因：

DNS（域名系统）、SMTP（邮件发送）、TFTP（简单文件传输）、FTP（文件传输）、HTTP（超文本传输）、Telnet（远程登录）

要求：

1. 列出每个协议所使用的传输层协议（TCP 或 UDP）
2. 简述选择该传输层协议的原因
3. 对比分析：为何有些协议使用 TCP，而有些使用 UDP？

参考答案

应用层协议	传输层协议	原因
DNS	UDP (通常)、TCP (特定情况)	查询报文小, 无连接开销低; 区域传输和大报文时用 TCP
SMTP	TCP	邮件内容重要, 需要可靠传输和顺序保证
TFTP	UDP	简单协议, 快速传输, 自己处理重传, 无需连接开销
FTP	TCP	文件传输需可靠性, 建立控制连接和数据连接
HTTP	TCP	Web 内容重要, 需可靠传输, HTTPS 基于 TLS 进一步增强
Telnet	TCP	远程交互需可靠性和顺序性

关键分析:

1. 使用 TCP 的协议 (FTP、SMTP、HTTP、Telnet) :

- 共同点: 数据内容对应用很重要, 丢失会造成严重后果
- 特点: 都建立面向连接的通信, 确保可靠性
- 代价: 连接建立、确认、重传等开销较大

2. 使用 UDP 的协议 (DNS、TFTP) :

- 共同点: 报文量小, 可以容忍偶尔丢包
- 特点: 无连接, 直接发送, 延迟低
- 优势: 减少开销, 适合查询类应用

3. UDP 与 TCP 的权衡:

- DNS 通常用 UDP (报文 ≤ 512 B) , 但区域传输必须用 TCP
- TFTP 用 UDP 但自己实现重传机制
- 选择标准: 可靠性需求 vs 响应速度和开销

3. http案例 (12分)

下面的文本显示服务器对上述HTTP GET消息的回复。回答以下问题, 指出在下面的消息中找到答案的位置。

```
HTTP/1.1 200 OK<cr><lf>Date: Tue, 07 Mar 2008 12:39:45GMT<cr><lf>Server:
Apache/2.0.52 (Fedora) <cr><lf>Last-Modified: Sat, 10 Dec2005 18:27:46 GMT<cr>
<lf>ETag: "526c3-f22-a88a4c80"<cr><lf>AcceptRanges: bytes<cr><lf>Content-Length:
3874<cr><lf> Keep-Alive: timeout=max=100<cr><lf>Connection: Keep-Alive<cr>
<lf>Content-Type: text/html; charset= ISO-8859-1<cr><lf><cr><lf><!doctype html
public "-//w3c//dtd html 4.0transitional//en"><lf><html><lf> <head><lf> <meta
http-equiv="Content-Type" content="text/html; charset=iso-8859-1"><lf> <meta
name="GENERATOR" content="Mozilla/4.79 [en] (windows NT 5.0; U) Netscape]"><lf>
<title>CMPSCI 453 / 591 / NTU-ST550Aspring 2005 homepage</title><lf></head><lf>
<much more document text following here (not shown)>
```

- a. 服务器是否成功找到了文档？提供了什么时间的文档回复？
- b. 文档最后一次修改是什么时候？
- c. 返回的文档中有多少字节？
- d. 返回的文档的前5字节是什么？服务器是否同意持久连接？

答案：

- a. 是的，服务器成功找到了文档（HTTP/1.1 200 OK）。回复时间是：Tue, 07 Mar 2008 12:39:45 GMT（来自Date头）
- b. 最后修改时间：Sat, 10 Dec 2005 18:27:46 GMT（来自Last-Modified头）
- c. 返回3874字节（来自Content-Length: 3874头）
- d. 前5字节是：<!doc（来自响应体的<!doctype...开始）。是的，服务器同意持久连接（Connection: Keep-Alive）

4. HTTP 协议进化（8分）

HTTP/1.1中的HOL阻塞问题是什么？HTTP/2如何尝试解决它？

答案：

HOL(Head-of-Line)阻塞问题：

- 在HTTP/1.1中，浏览器通过TCP连接发送请求，必须等待该请求的响应完整到达后，才能发送下一个请求
- 如果某个大对象的响应传输很慢，后续小对象的请求会被阻塞
- 这导致了较低的网络利用率

HTTP/2的解决方案：

- HTTP/2使用多路复用(multiplexing)机制
- 在单个TCP连接上，可以同时发送多个HTTP请求，每个请求和响应都有唯一标识符
- 响应可以交错到达，接收方根据标识符重新组装
- 这消除了HOL阻塞问题，提高了网络利用率

5. TCP问题（10分）

假设主机A连续向主机B通过TCP连接发送两个TCP段。第一个段的序列号为90；第二个段的序列号为110。

a) 第一个段中有多少数据？

b) 假设第一个段丢失但第二个段到达B。在主机B发送给主机A的确认中，确认号是什么？

答案

a) 第一个段中的数据字节数：

字节数 = 第二个段的序列号 - 第一个段的序列号
= 110 - 90
= 20 字节

b) 第二个段丢失时的确认号：

- **情况分析：**

1. 第一个段（序列号90-109）丢失了
2. 第二个段（序列号110-129）到达了
3. 主机B接收到了"期望的序列号不连续"的数据

- **主机B的行为**（根据TCP的接收规则）：

1. 主机B期望的下一个字节是序列号90的数据
2. 但收到的是序列号110的数据
3. 由于有"间隔"，主机B会：
 - 缓存序列号110-129的数据
 - 发送确认号为 **90** 的ACK（表示"我期望收到序列号为90的字节"）

- **主机A收到这个ACK的含义：**

- 发现序列号90的段未被确认
- 意识到序列号90的段丢失了
- 将使用超时重传机制重新发送这个段

6. TCP和UDP协议区别（6分）

说明为什么应用开发者可能选择在UDP上而不是TCP上运行应用。

答案：

主要原因：

1. **对拥塞控制的回避：**UDP不实现TCP的拥塞控制，应用可以以恒定速率发送，不会被TCP的速率限制
2. **低延迟：**UDP没有TCP的三次握手和连接建立开销，可以立即开始传输数据
3. **应用层控制：**开发者对何时、如何发送数据有更精细的控制
4. **实时应用适配：**对于IP电话、视频会议等应用，偶尔丢失的数据包比延迟更可接受
5. **多播/广播支持：**UDP支持多播和广播，TCP不支持

7. UDP（7分）

应用程序是否可能在基于UDP的应用上享受可靠数据传输？如果可以，怎样实现？

答案：

是的，完全可能。具体实现方法：

1. **应用层实现：**应用程序可以在应用层自己实现可靠性机制，包括：
 - 序列号：为每个UDP消息添加序列号

- 确认机制：接收者确认收到的消息
 - 超时重传：发送者对未确认的消息进行重传
 - 去重：处理可能的重复消息
2. **技术复杂性**：这需要大量的编程工作，本质上是在应用层重新实现TCP的可靠性功能
3. **实际例子**：QUIC协议就是这样做的——在应用层实现了可靠性、流控制和拥塞控制

8. 可靠数据传输（8分）

传输层协议中为什么需要引入序列号和定时器

答案：

一、序列号的关键作用

1. 重复检测

- 接收方可以判断接收到的数据包是否是重复的（之前已经接收过的数据包）
- 即使同一个数据包被多次接收，接收方也能通过序列号识别出是重传

2. 乱序检测

- 可以判断数据包是否按照发送顺序到达
- 帮助接收方维持数据的正确顺序

3. 丢失检测

- 通过序列号的间隔可以检测是否有数据包丢失
- 例如收到序列号为 0 和 2 的包，可以判断序列号为 1 的包丢失了

在 RDT 协议中的具体应用：

- 当发送方因为超时而重传数据包时，接收方需要通过序列号判断这是新数据还是重传的数据
- 没有序列号，接收方无法区分一个新的数据包和一个重传的数据包，可能导致数据重复接收

二、定时器的关键作用

1. 丢失检测

- 当发送方发送数据包后，如果在指定时间内没有收到确认（ACK），说明数据包或确认可能已丢失
- 超时判断是否需要重传的唯一可靠方法

2. 自动重传

- 超时时，发送方自动重传该数据包
- 确保即使网络丢失数据也能最终传递成功

3. 保证可靠性

- 即使信道会丢失数据包，通过定时器的超时重传机制，最终也能确保数据正确送达
- 使得不可靠的底层网络提供可靠的传输保证

没有定时器的后果：

- 发送方发送数据包后无法知道是否成功传递
- 如果确认丢失，数据包也不会被重新发送

- 通信可能无限期地等待，永远无法完成

9. FTP 协议与 TCP 拥塞控制 (22分)

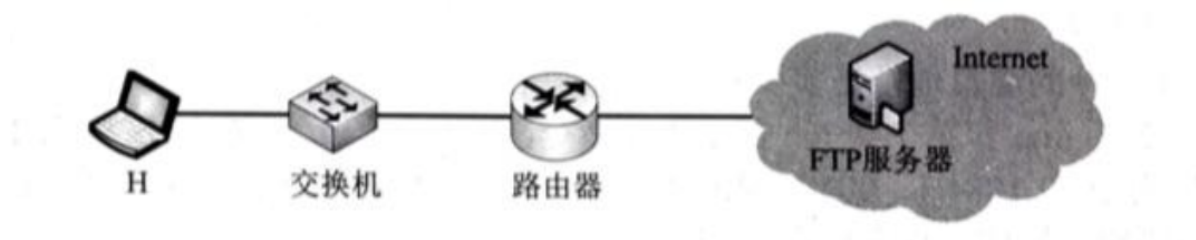
某网络拓扑如下图所示，主机 H 登录 FTP 服务器后，向服务器上传一个大小为 18000 B 的文件 F。假设 H 为传输 F 建立数据连接时，选择的初始序号为 100， $MSS = 1000\text{ B}$ ，拥塞控制初始阈值为 4 MSS， $RTT = 10\text{ ms}$ ，忽略 TCP 段的传输时延；在 F 的传输过程中，H 均以 MSS 段向服务器发送数据，且未发生差错、丢包和乱序现象。

(1) FTP 的控制连接是持久的还是非持久的？FTP 的数据连接是持久的还是非持久的？H 登录 FTP 服务器时，建立的 TCP 连接是控制连接还是数据连接？（6分）

(2) H 通过数据连接发送 F 时，F 的第 1 个字节的序号是多少？在断开数据连接的过程中，FTP 服务器发送的第二次挥手 ACK 段的确认序号是多少？（6分）

(3) H 通过数据连接发送 F 时，当 H 收到确认序号为 2101 的确认段时，H 的拥塞窗口调整为多少 MSS？当 H 收到确认序号为 7101 的确认段时，H 的拥塞窗口调整为多少 MSS？（6分）

(4) H 从请求建立数据连接开始，到确认 F 已被服务器全部接收为止，至少需要多长时间，期间应用层数据平均发送速率是多少？（4分）



参考答案

(1)

- FTP 的控制连接是持久的
- FTP 的数据连接是非持久的
- H 登录 FTP 服务器时，建立的 TCP 连接是控制连接

(2)

F 第 1 个字节的序号：

在建立数据连接时，H 发送 SYN 段，其序号为 100（初始序号）。SYN 段本身占用 1 个序号，三次握手完成后，数据传输从序号 101 开始。

F 第 1 个字节的序号 = 101

第二次挥手的确认序号：

F 文件共 18000 B，最后一个字节的序号为：

$$101 + 18000 - 1 = 18100$$

H 数据发送完后，发送 FIN 段，其序号为 18101。FTP 服务器在第二次挥手时回复 ACK 段，确认序号为：

$$18101 + 1 = 18102$$

(3)

拥塞窗口变化分析：

初始条件：ssthresh = 4 MSS, cwnd 从 1 MSS 开始

慢开始阶段 ($cwnd < ssthresh$)：每收到一个 ACK, $cwnd + 1$ MSS

拥塞避免阶段 ($cwnd \geq ssthresh$)：每个 RTT, $cwnd + 1$ MSS

逐步推导：

轮次	发送前 cwnd	发送数据段序号	收到 ACK	cwnd 变化	阶段
1	1 MSS	101~1100	1101	→ 2 MSS	慢开始
2	2 MSS	1101~2100、2101~3100	3101	→ 4 MSS	转拥塞避免
3	4 MSS	3101~4100、4101~5100、5101~6100、6101~7100、...	7101	→ 5 MSS	拥塞避免

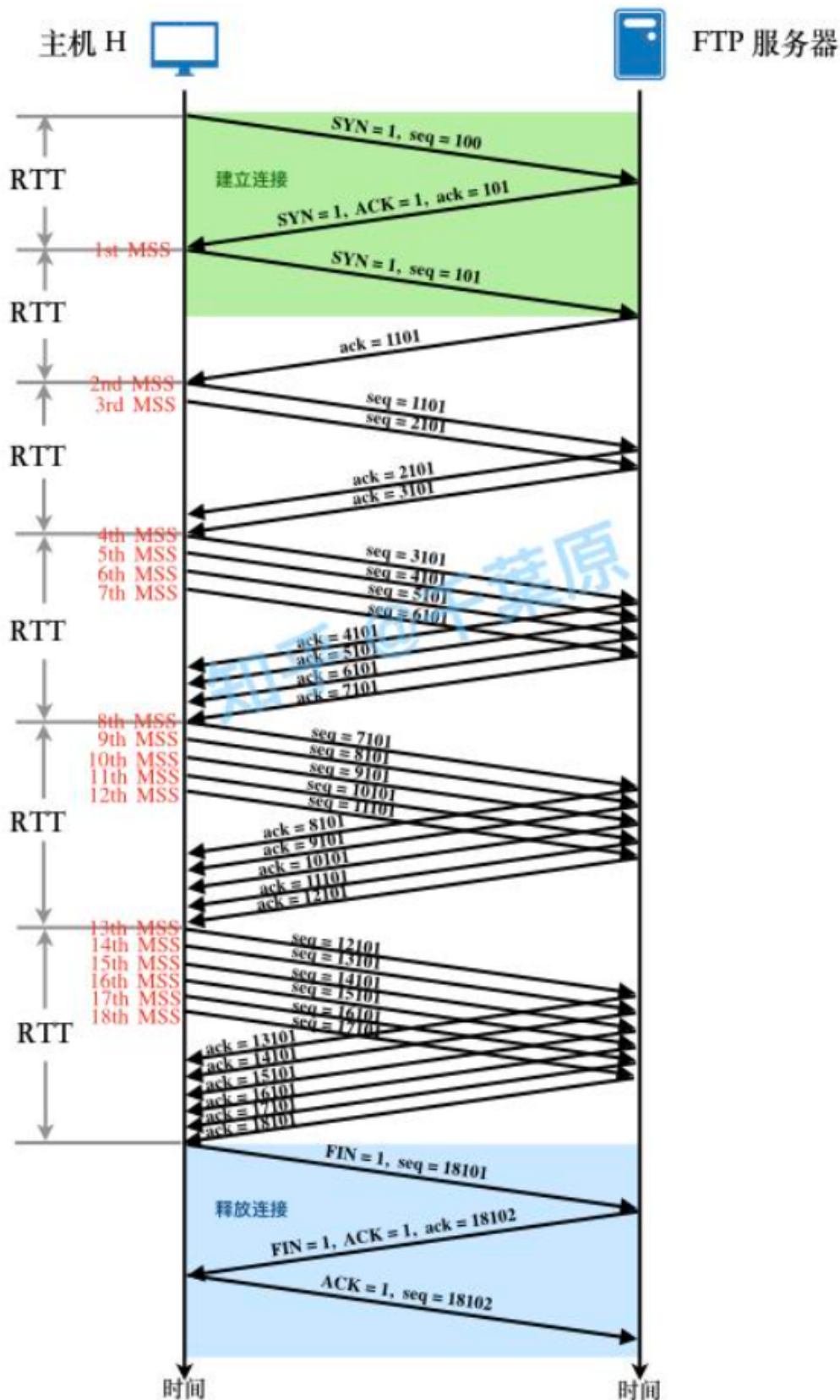
答案：

- 收到 ACK = 2101 时, $cwnd = 3$ MSS
- 收到 ACK = 7101 时, $cwnd = 5$ MSS

(4)

根据 (3) 的分析，从请求建立数据连接开始，到确认 F 已被服务器全部接收为止，需要经过 6 个 RTT，其中第 1 个 RTT 用于建立连接的第一次和第二次握手，第三次握手携带数据，后 5 个 RTT 用于传输数据。

为了简化模型，假设同一个 TCP 段的发送和确认在同一个 RTT 内完成，可画出如下示意图。



因为RTT=10ms, 6RTT=60ms, H从请求建立数据连接开始, 到确认F已被服务器全部接收为止, 至少需要60ms。

期间应用层传输数据量为文件F的大小18000B, 根据第一问的分析, 该传输过程至少需要60ms。因此, 应用层数据平均发送速率是18000B/60ms=2.4Mbps。

